INTERNSHIP REPORT

ON

FULL STACK DEVELOPMENT, SOFTWARE TESTING AND ENTERPRISE APPLICATION DEVELOPMENT

With emphasis on SaaS Task Management and School ERP Examination Systems

Submitted to

School of Computer Science and Engineering
IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of
B.Tech CSE (AI/ML)

By

RAMBHU SINGH

Roll Number: CS-23411331
Batch: 2023–27

2026–27

CANDIDATE'S DECLARATION

I, RAMBHU SINGH, do hereby declare that the internship report titled "INTERNSHIP REPORT ON FULL STACK DEVELOPMENT, SOFTWARE TESTING AND ENTERPRISE APPLICATION DEVELOPMENT" has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in CSE. The work described in this report represents my internship learning and project experience. References and supporting materials used in preparing the report have been acknowledged appropriately. This report has not been submitted to any other institute for any degree or diploma requirement. I shall be responsible for any copyright violation arising from material submitted by me.

Signature: _____

Student Name: RAMBHU SINGH

Roll No.: CS-23411331

ACKNOWLEDGEMENT

I express my sincere gratitude to Physics Wallah (FinZ) for providing me with the opportunity to undertake a three-month internship in the Development department as a Full Stack Developer. The internship provided practical exposure to software development, web application workflows, testing, debugging, and enterprise application modules.

I am particularly thankful to Mr. Piyush Jaiswal, National Head, for his professional guidance and for providing an environment in which I could learn through practical project responsibilities. I also acknowledge the support of the teams and colleagues who helped me understand application workflows, requirements, debugging practices, and software quality expectations.

I am grateful to IILM University, Greater Noida, and the School of Computer Science and Engineering for encouraging industry-oriented learning as part of the B.Tech CSE curriculum. I also express my gratitude to my parents and family for their continuous encouragement and support.

Signature: _____

Student Name: RAMBHU SINGH

Roll No.: CS-23411331

CERTIFICATE

OF COMPLETION

THIS IS TO CERTIFY THAT

Rambhu Singh

has successfully completed an internship as **Full Stack Developer** in **FinZ** from **18-May-2026** to **18-Aug-2026** .

During the internship, they demonstrated dedication, professionalism, and a willingness to learn. We appreciate their contribution and wish them success in their future endeavors.

30-Aug-2026
DATE

SIGNATURE

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

4. PROJECT DESCRIPTION

This chapter presents the practical work undertaken during the internship period from 18 May 2026 to 18 August 2026 at Physics Wallah (FinZ), along with an additional individual technical project developed outside the internship. The internship role was Full Stack Developer in the Development department. The internship was conducted in hybrid mode and focused on practical software engineering activities involving application workflows, backend development, testing, debugging, and enterprise software modules.

The internship work discussed in this report primarily covers two applications: the Shambhavika clothing-brand application, where the major responsibility was application testing and workflow validation, and a SaaS-based school ERP system, where responsibility was concentrated on offline and online examination functionality, scheduling, information management, validation, and reliability. FlowEscalate is presented separately as an individual technical project to demonstrate additional full-stack development capability.

The internship offer letter records the organization as Physics Wallah (FinZ) and states that the student would join the Development department as a Full Stack Developer for a three-month term beginning on 18-05-2026.

4.1 Introduction

The internship was undertaken to bridge the gap between academic knowledge and production-oriented software engineering practice. Before the internship, the student had exposure to Java, Python, C/C++, HTML/CSS, JavaScript, React, Spring Boot, SQL, DSA, and AI/ML. During the internship, this foundation was extended into PHP and Laravel based full-stack development, web application testing, enterprise workflows, database-backed modules, REST-oriented application interaction, version control, and systematic debugging.

A significant part of the learning experience came from working with existing applications rather than isolated academic programs. This required understanding requirements, navigating a codebase, reproducing issues, validating business rules, checking edge cases, and ensuring that a change in one workflow did not introduce failures elsewhere. Such activities are central to professional software engineering because correctness is determined not only by whether code compiles, but also by whether the complete user workflow behaves as intended.

The Shambhavika application provided an e-commerce style environment for validating customer-facing workflows. The School ERP application provided a

broader enterprise environment in which examination schedules, question banks, marks-related workflows, offline examinations, and online examinations had to remain consistent and reliable. These experiences strengthened practical understanding of functional testing, UI validation, form validation, debugging, responsive behaviour, and end-to-end workflow verification.

The additional FlowEscalate project demonstrates independent implementation of a multi-tenant SaaS platform. Its architecture includes tenant isolation, role-based access control, task management, SLA monitoring, workload visibility, and audit logging. It is not presented as Physics Wallah (FinZ) internship work; instead, it is included as an individual technical project that complements the internship experience.

4.1.1 Internship Profile

The offer letter also specifies that the internship was voluntary and subject to company policies, and that work products created during the term could be treated as company property. These terms reinforce the importance of confidentiality and responsible handling of project information.

Table 4.1 — Internship profile and role

Parameter	Details
Student	RAMBHU SINGH
Roll Number	CS-23411331
University	IILM University, Greater Noida
Programme	B.Tech CSE (AI/ML)
Batch	2023–27
Organization	Physics Wallah (FinZ)
Role	Full Stack Developer
Internship Period	18-05-2026 to 18-08-2026
Working Mode	Hybrid
Industry Mentor	Piyush Jaiswal, National Head
Stipend	₹0 per month as stated in offer letter

4.1.2 Role Transition and Learning Path

The learning path during the internship can be viewed as a transition from technology familiarity to application-level responsibility. The student already possessed a broad academic foundation in programming, web technologies, databases, DSA, and AI/ML. The internship added practical depth in PHP and Laravel, MVC-oriented development, Blade templating, Tailwind CSS, Vite-based frontend bundling, REST API interaction, database-backed workflows, testing, debugging, and Git/GitHub based development practices.

An important lesson was that full-stack development is not limited to implementing isolated screens. A feature generally crosses several layers: user interface, request handling, validation, business logic, persistence, authorization, error handling, and testing. Working on enterprise applications helped develop the ability to trace a requirement through these layers and verify the complete workflow.

The internship also strengthened communication and problem-solving skills. Testing required writing reproducible observations, identifying the exact point of failure, checking whether the behaviour matched the expected workflow, and confirming the fix through retesting. Examination modules required special attention because incorrect scheduling, incomplete data, or inconsistent validation can directly affect students and institutional operations.

4.2 Organization Profile

Physics Wallah (FinZ), also referred to in the offer letter as ?Physics Wallah (FinZ)? or ?Physics Wallah?, provided the internship opportunity in its Development department. The offer letter identifies the corporate location as Noida, Uttar Pradesh, and records the corporate address as Plot No. B-8, Tower A, Noida One, Ground Floor / First Floor, Sector 62, Noida, UP ? 201309.

The internship role was Full Stack Developer, which provided an opportunity to work across application layers and understand the lifecycle of software from requirements and implementation to testing and debugging. The work environment exposed the student to multiple application domains, including consumer-facing e-commerce workflows and education-focused enterprise resource planning.

Because the internship involved company and project information, confidentiality was an explicit requirement. The offer letter states that confidential, proprietary, and trade-secret information accessed during the internship must be kept confidential.

The organization context therefore influenced the working approach: application data had to be treated carefully, changes had to be validated before being considered complete, and project-specific information had to be used responsibly. The report consequently focuses on technical responsibilities and publicly presentable screenshots rather than confidential internal information.

4.2.1 Internship Working Environment

The internship followed a hybrid working mode. A hybrid environment requires disciplined communication and self-management because development, testing, debugging, and issue verification may occur across different working locations. The

student had to maintain continuity between tasks, reproduce issues consistently, and communicate the status of work clearly.

Working on existing applications also required adapting to established naming conventions, project structure, workflows, and technology choices. Rather than creating every component from scratch, the student learned to understand an existing codebase, identify relevant modules, make focused changes, and validate their effect on related functionality.

The internship offer letter specifically required the intern to work using a personal laptop rather than a company-provided laptop. This reflects the practical nature of the internship arrangement and reinforced the need for a dependable local development and testing environment.

4.3 Problem Statement

Modern business and educational applications contain interconnected workflows where a small defect can affect multiple downstream operations. In a clothing e-commerce application, an incorrect validation or broken cart transition can prevent customers from completing purchases. In a school ERP, an error in examination scheduling or online examination submission can affect a large number of users and create operational difficulties for administrators and students.

The central problem addressed through the internship was therefore not a single coding defect, but the broader requirement to build and validate reliable application workflows. The work required ensuring that user actions were correctly validated, that data moved through the expected states, and that modules remained usable across normal and edge-case scenarios.

For the Shambhavika application, the testing problem focused on validating the end-to-end customer journey: authentication, product discovery, product interaction, cart operations, checkout, responsiveness, and general performance. For the School ERP, the problem was more data-sensitive: examination types, schedules, grades, marks setup, question-bank workflows, offline examinations, and online examinations needed to operate without functional inconsistencies.

The individual FlowEscalate project addresses a different engineering problem: organizations using a shared SaaS platform require strict tenant isolation, controlled access by role, task assignment, SLA monitoring, escalation, and auditability. The project demonstrates how these requirements can be addressed through application architecture.

4.3.1 Testing and Reliability Challenges

Testing enterprise applications requires more than checking whether a page loads. A reliable test process asks whether the correct user can access the correct functionality, whether invalid input is rejected, whether valid input is persisted, whether transitions happen in the correct order, and whether the system remains stable after repeated actions.

UI testing adds another dimension. Text, buttons, forms, cards, navigation elements, and responsive layouts must remain understandable and usable. A feature can be logically correct while still creating a poor user experience if elements overlap, controls become inaccessible, or feedback is unclear.

Performance testing and practical responsiveness checks were also part of the Shambhavika testing responsibilities. The objective was not to perform a formal benchmark study, but to observe whether the application remained reasonably responsive during normal workflows and whether obvious bottlenecks or broken interactions were present.

The examination system introduced additional correctness constraints. Exam schedules and online examination flows require careful handling of timing, access, submission, and question data. The student therefore treated correctness and regression checking as important parts of the development/testing cycle.

4.4 Project Objectives

1. Understand and apply a professional full-stack development workflow using PHP and Laravel.
2. Gain practical experience with MVC-oriented application development, database-backed modules, validation, and REST-oriented interactions.
3. Perform structured functional, UI, form, authentication, product, cart, checkout, responsive, performance, and regression-oriented testing.
4. Support the development and validation of offline and online examination workflows within a SaaS-based school ERP.
5. Ensure examination scheduling and examination information workflows behave consistently and accurately.
6. Develop the ability to reproduce, document, debug, retest, and close software defects.
7. Strengthen understanding of enterprise application concerns such as authorization, data integrity, workflow states, and auditability.
8. Develop an independent technical project demonstrating multi-tenant SaaS architecture and automated SLA escalation.

4.4.1 Personal Contribution

Table 4.2 — Major project responsibilities

Project / Area	Primary responsibility	Nature of work
Shambhavika	Testing and workflow validation	Functional, UI, form, authentication, product, cart, checkout, responsive and performance-oriented manual testing; bug reporting and debugging.
School ERP Examination modules	Examination modules	Offline and online examination functionality, exam scheduling, question-bank related workflows, marks/examination information, validation, testing and reliability checks.
FlowEscalate	Independent full-stack development	Laravel/PHP backend architecture, tenant isolation, RBAC, availability/team escalation workflows, Blade/Tailwind frontend integration, PHPUnit tests.

4.4.2 Learning Objectives Achieved

The internship achieved the objective of converting theoretical knowledge into practical engineering habits. The student learned that development and testing are iterative activities: a feature is implemented, executed through realistic workflows, checked against expected behaviour, corrected where necessary, and tested again. This cycle was repeatedly applied to both consumer-facing and enterprise applications.

The experience also strengthened attention to detail. In examination workflows, the correctness of dates, schedules, question availability, access conditions, and submission behaviour is important. In e-commerce workflows, the continuity from product discovery to cart and checkout is essential. These differences helped the student understand how domain requirements shape software validation.

4.5 Scope of the Project

The scope of the internship work includes practical software development and testing activities across two application domains. The first is a clothing-brand application in which the student focused primarily on testing the user journey and validating that the application behaved correctly. The second is a SaaS-based school ERP system in which the student handled examination-related functionality, including offline and online examinations and scheduling workflows.

The scope includes manual testing, UI validation, form validation, login/signup validation, product workflows, cart and checkout workflows, bug reporting, debugging, responsive checks, performance-oriented checks, and regression validation. In the school ERP, the scope extends to examination configuration, schedules, marks-related setup, question-bank workflows, offline examination management, online examination management, and related information flows.

FlowEscalate is within the scope of this report as an additional individual technical project. It demonstrates multi-tenant SaaS architecture, role-based access, task

management, SLA monitoring, team workload, audit logging, and escalation concepts. It is intentionally separated from the Physics Wallah (FinZ) internship project to maintain factual accuracy.

4.5.1 Shambhavika — Application and Testing Scope

The Shambhavika application presents an e-commerce style clothing experience. The supplied application screen shows a branded home page with search, promotional content, product categories, product cards, and navigation elements for Home, Explore, Cart, Orders, and Profile. The testing responsibility was to verify that the user-facing workflow operated correctly rather than to redesign the application.

Testing covered functional behaviour, UI consistency, form validation, login/signup, product interaction, cart, checkout, responsive behaviour, and performance-oriented observations. Manual testing was used to execute realistic user flows and identify defects. When a defect was found, the workflow was reproduced, the problem was isolated, corrected where applicable, and retested.

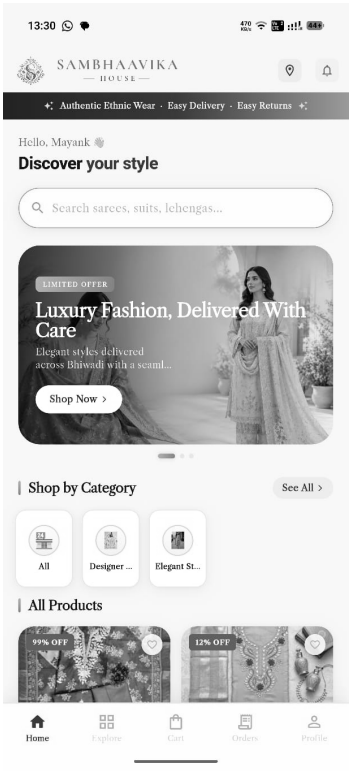

Figure 4.1 — Shambhavika home screen used for workflow validation

4.5.2 Shambhavika — Representative Test Areas

Table 4.3 — Functional modules of Shambhavika

Test Area	Validation focus	Expected result
Authentication	Login/signup fields and valid/invalid inputs	Correct authentication response and validation feedback.
Home/UI	Navigation, search, cards, categories, banners	Elements visible and interactive without obvious layout defects.
Products	Product selection and product information	Correct product information and navigation.
Cart	Add/remove/update cart behaviour	Cart reflects the user’s intended selections.
Checkout	Required fields and workflow transitions	Valid information allows progression; invalid data is rejected.
Responsive UI	Different viewport/device sizes	Content remains accessible and usable.
Performance	Normal workflow responsiveness	No obvious blocking or broken interactions during ordinary use.

4.5.3 School ERP — Application Scope

The School ERP application, shown in the supplied screenshots as the EduStaff Portal / School ERP System, provides a dashboard with modules such as Enquiries, Student Information, Academics, Study Center, Staff Attendance, Finance, Examinations, Online Exams, and Library. The student's work focused on the examination-related portion of the system.

The application includes separate navigation groups for Offline Examinations and Online Examinations. The Offline Examinations group exposes Manage Offline Exams, Exam Types, Manage Grades, Cocurricular Areas, Schedule & Marks Setup, Enter Marks, and related marksheet/admit-card functions. The Online Examinations group exposes Manage Online Exams and Question Bank.

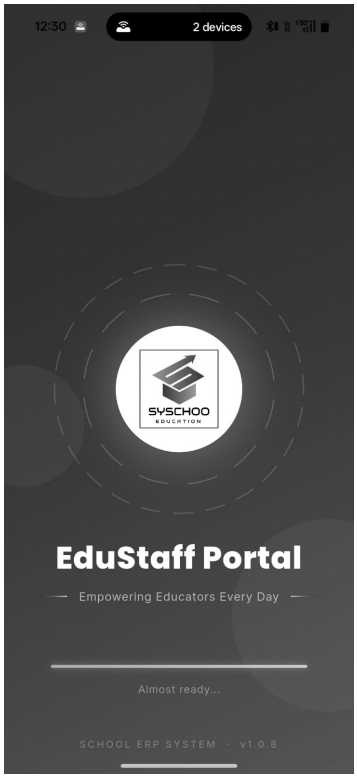

Figure 4.2 — EduStaff Portal splash screen

4.5.4 School ERP — Offline Examination Scope

The offline examination responsibility covered the complete functional area assigned to the student. This included managing examinations, examination types, grades, examination schedules, marks setup, marks entry, and supporting information used for marksheets and admit cards. The objective was to keep the data flow accurate from configuration through schedule and marks-related operations.

Examination scheduling requires careful validation of dates, times, academic groups, and the relationship between an examination and the students or classes affected by it. A scheduling defect can create conflicts or incorrect examination information. Therefore, testing focused on both valid workflows and boundary conditions such as missing information, inconsistent selections, and repeated edits.

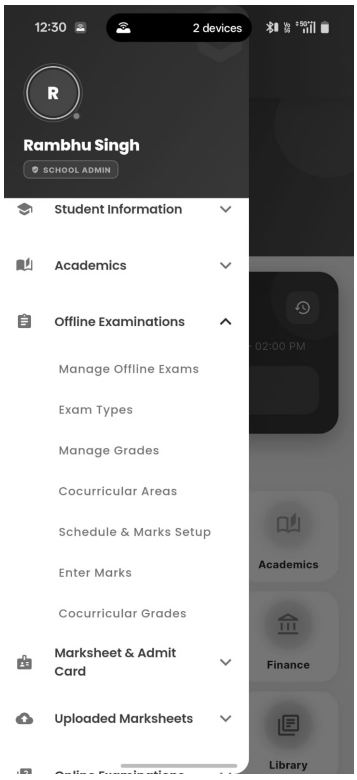

Figure 4.3 — Offline Examination navigation and related academic options

4.5.5 School ERP — Online Examination Scope

The online examination module provides a digital route for conducting tests. The student's responsibility included managing and validating the online examination workflow and its supporting question-bank functionality. The aim was to ensure that examinations could be configured correctly and that the student-facing examination process remained usable and reliable.

Online examination workflows introduce additional correctness concerns such as question availability, exam access, timing, answer interaction, navigation, and submission. Testing therefore emphasized complete user journeys and careful

verification of transitions. The student also checked for code-level errors and workflow failures so that an online test could be attempted effectively without avoidable application errors.

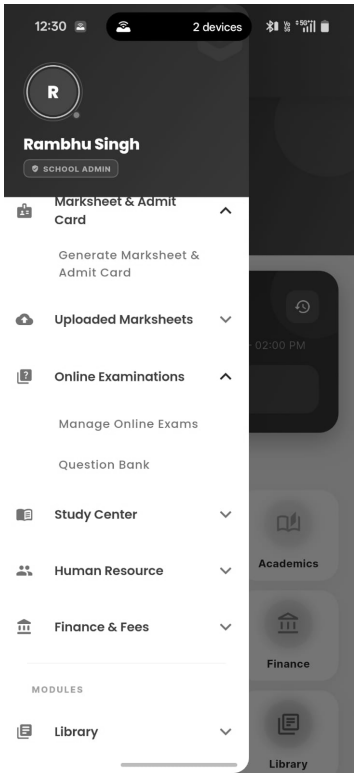

Figure 4.4 — Online Examination navigation showing Manage Online Exams and Question Bank

4.5.6 School ERP — Examination Responsibilities

Table 4.4 — School ERP examination responsibilities

Responsibility	Description
Exam management	Create, manage and validate examination-related records and workflows.
Exam types and grades	Maintain supporting academic configuration used by examinations.
Scheduling	Manage examination schedules and validate relevant information.
Marks setup	Validate marks-related configuration and data flow.
Marks entry	Check that marks can be entered through the intended workflow.
Marksheet/admit card support	Validate examination data required by related documents.
Online examinations	Validate digital examination workflow and associated configuration.
Question bank	Work with the question-management area supporting online examinations.
Testing/debugging	Reproduce defects, verify fixes and perform regression checks.

4.6 Technologies and Tools Used

The internship introduced practical use of PHP and Laravel as the principal web-development stack. Supporting technologies and tools included HTML, CSS, JavaScript, Blade templates, Tailwind CSS, Vite, Composer, database systems, REST-oriented application interfaces, Git/GitHub, XAMPP/local development tooling, and testing/debugging facilities available through the development environment.

The exact supporting technology can vary by project module. The common engineering principle was to use the framework and supporting tools already established by the application while maintaining consistency with the project structure. This helped the student learn how professional development differs from isolated coding exercises.

Table 4.5 — Technology stack

Technology / Tool	Purpose in practical work
PHP	Server-side application development and business logic.
Laravel	MVC-oriented framework, routing, controllers, validation, services and application structure.
Blade	Server-rendered view templates and UI composition.
HTML/CSS	Structure and presentation of web interfaces.
JavaScript	Client-side interaction and dynamic behaviour.
Tailwind CSS	Utility-based styling and responsive UI implementation.
Vite	Frontend asset bundling and development workflow.
Composer	PHP dependency management.
MySQL / SQLite	Relational persistence depending on project/environment.
REST API	Application-to-application or client-server interaction patterns.
Git/GitHub	Version control, source management and project collaboration.
XAMPP	Local web-server/development environment where applicable.
PHPUnit	Automated unit/feature testing for the independent FlowEscalate project.

4.6.1 PHP and Laravel

PHP provided the server-side programming foundation for the web applications. Laravel provided a structured framework for routing, controllers, models, validation, middleware, database access, views, services, commands, and testing. Learning Laravel was significant because it encouraged separation of responsibilities and made it easier to trace an incoming request through the application layers.

The student developed familiarity with MVC principles: routes define entry points, controllers coordinate requests, models represent data, views present information, and services or supporting classes can encapsulate reusable business logic. This architecture improved maintainability and made debugging more systematic.

The independent FlowEscalate project used PHP 8.2 and Laravel and included backend architecture, tenant isolation, RBAC, user availability and escalation workflows, Blade templates, Tailwind CSS, Vite, and PHPUnit test suites. This project is explicitly separated from the Physics Wallah (FinZ) internship work.

4.6.2 Frontend Technologies

HTML and CSS were used as the structural and presentation foundation. Blade templates enabled server-rendered Laravel views, while Tailwind CSS provided a consistent utility-based styling approach. JavaScript supported client-side interactions where required. Vite was used for frontend asset bundling in the independent FlowEscalate project.

The internship helped reinforce the importance of frontend correctness. Testing was not restricted to backend results; the student checked whether users could see, understand, and interact with controls. Responsive testing was especially relevant for the Shambhavika application because the supplied screen demonstrates a mobile-oriented shopping interface.

4.6.3 Databases and APIs

Database-backed applications require careful handling of persistence, validation, and relationships. The projects involved relational database concepts and database

access through application frameworks. The student gained practical exposure to storing and retrieving application information, validating data before persistence, and checking whether updates were reflected correctly in dependent workflows.

REST-oriented interfaces were also part of the full-stack learning path. Understanding request/response behaviour helped connect frontend interactions with backend processing and reinforced the importance of validation, error handling, and predictable data formats.

4.7 System Architecture

The internship applications can be understood through a layered architecture consisting of presentation, application logic, persistence, and testing/quality layers. The presentation layer contains user-facing screens and interactions. The application layer handles routing, authorization, validation, business rules, and workflow transitions. The persistence layer manages structured data. The testing layer verifies expected behaviour and protects against regressions.

For the School ERP, the architecture is especially important because many modules share common academic data. Examination schedules, grades, marks, questions, students, and related records must remain consistent. A change in one module can affect another, so testing must consider integration points rather than treating each page as independent.

For Shambhavika, the primary workflow is customer-centric: authentication → browsing → product interaction → cart → checkout → order-related workflow. Testing each transition helps identify failures that may not be visible when individual pages are tested in isolation.

4.7.1 Generic Full-Stack Request Flow

A typical full-stack request begins when a user interacts with a screen. The browser or client sends a request to the server. Routing identifies the appropriate endpoint. Middleware may authenticate the user or apply access rules. The controller receives the request and invokes validation and business logic. Data is read or updated through models/database access. The response is then rendered through a view or returned through an API response.

This flow provided a practical framework for debugging. When a UI action failed, the student learned to ask whether the problem originated in the view, request data, route, controller, validation, business logic, database, or response rendering. This layer-by-layer reasoning is one of the most valuable skills developed during the internship.

4.7.2 School ERP Examination Architecture

The School ERP examination area can be represented as a set of connected modules rather than a single page. Administrative configuration feeds examination setup; examination setup feeds schedules; schedules determine when and for whom an examination is available; marks and question-related data support the respective offline and online examination processes.

The offline path emphasizes institutional scheduling and marks workflows. The online path emphasizes digital exam access, questions, interaction, and submission. Both paths depend on accurate academic data and authorization. Therefore, validation and testing were treated as cross-module activities.

The screenshots supplied for the report show the application’s navigation hierarchy clearly. The School Admin dashboard exposes Examinations and Online Exams as distinct quick actions, while the side navigation further separates Offline Examinations from Online Examinations. This supports a modular architecture in which related functions are grouped by responsibility.

4.7.3 Shambhavika Architecture and Workflow

The Shambhavika application is organized around a shopping workflow. The home page contains search and category discovery, followed by product browsing and selection. The navigation includes Cart, Orders, and Profile, indicating a user journey that extends beyond simple product display.

From a testing perspective, the architecture can be viewed as a sequence of state transitions. A user must be able to authenticate, discover a product, inspect it, add it to a cart, review the cart, and proceed through checkout. Each transition must preserve relevant data and provide clear feedback. Testing the complete sequence therefore gives stronger confidence than testing each screen independently.

4.7.4 FlowEscalate — Individual Technical Architecture

FlowEscalate is an individual technical project developed separately from the Physics Wallah (FinZ) internship. It is a SaaS-oriented task management and SLA escalation platform. Its design focuses on multi-tenant data isolation, role-based access control, automated SLA monitoring, task assignment, team workload visibility, and auditability.

The backend was designed using PHP 8.2 and Laravel. Tenant context is applied so that organization-specific data remains isolated. Role-based access distinguishes administrative and operational responsibilities. Task workflows include assignment and status transitions, while SLA monitoring detects deadline breaches and supports escalation through an organizational hierarchy.

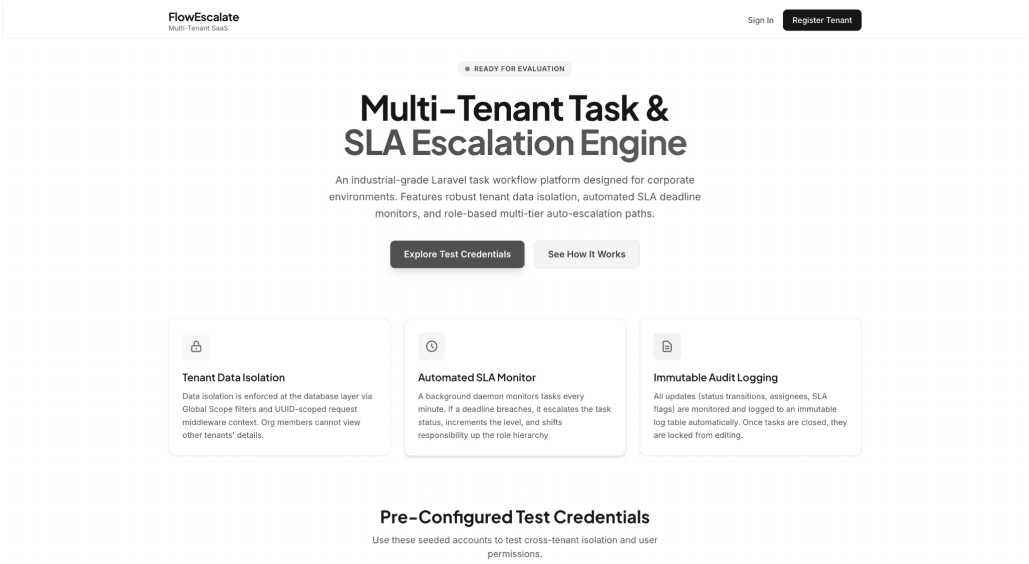

Figure 4.5 — FlowEscalate landing page showing multi-tenant task and SLA capabilities

4.7.5 FlowEscalate — Multi-Tenant Isolation

Multi-tenancy is a core architectural requirement. A shared application may host multiple organizations, but users from one organization must not be able to access another organization’s operational data. The project implements tenant-aware request context and database-level filtering using organization identifiers. This provides a defence-in-depth approach in which application logic consistently operates within the current tenant context.

The Super Admin view is intentionally different from organization dashboards. It provides a global perspective on tenants, subscriptions, and recurring revenue indicators. Organization users, by contrast, operate within their own tenant context. This separation demonstrates how SaaS systems combine global administration with isolated tenant operations.

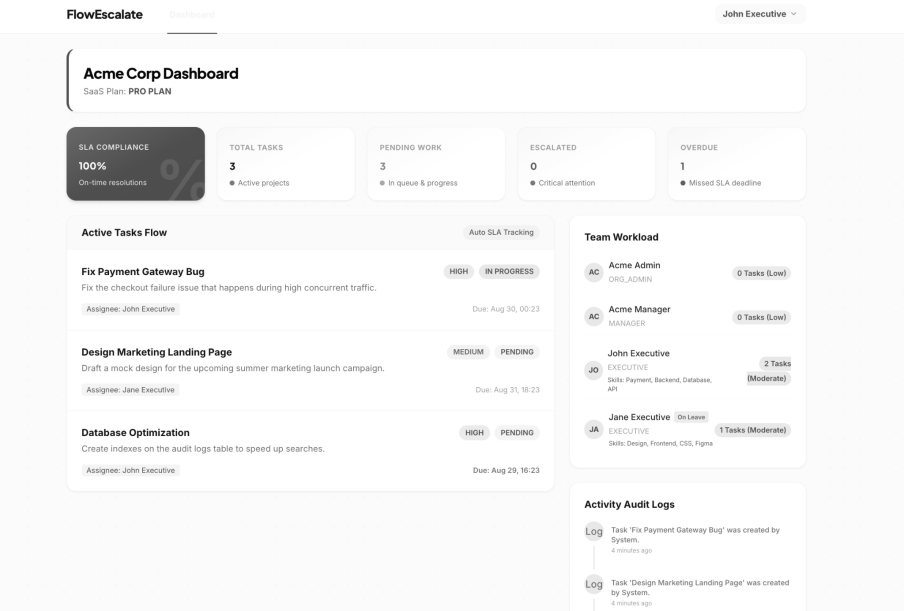

Figure 4.10 — FlowEscalate Super Admin dashboard for global tenant management

4.7.6 FlowEscalate — RBAC and Team Workload

The project uses role-based access control to separate responsibilities such as Super Admin, Organization Admin, Manager, and Executive. The supplied test credentials screen demonstrates separate roles across two organizations, but credentials are intentionally not reproduced in this report.

Team workload information supports task allocation by showing the number of assigned tasks and relevant skill information. This allows managers to consider availability and workload when assigning work. Audit information provides a chronological record of important task events, improving traceability.

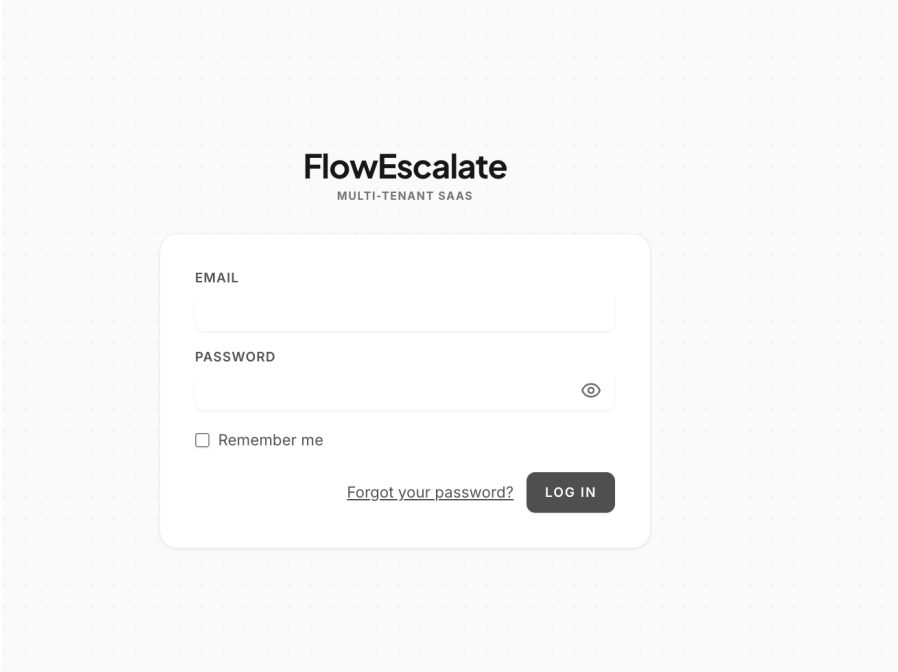

Figure 4.6 — FlowEscalate login screen

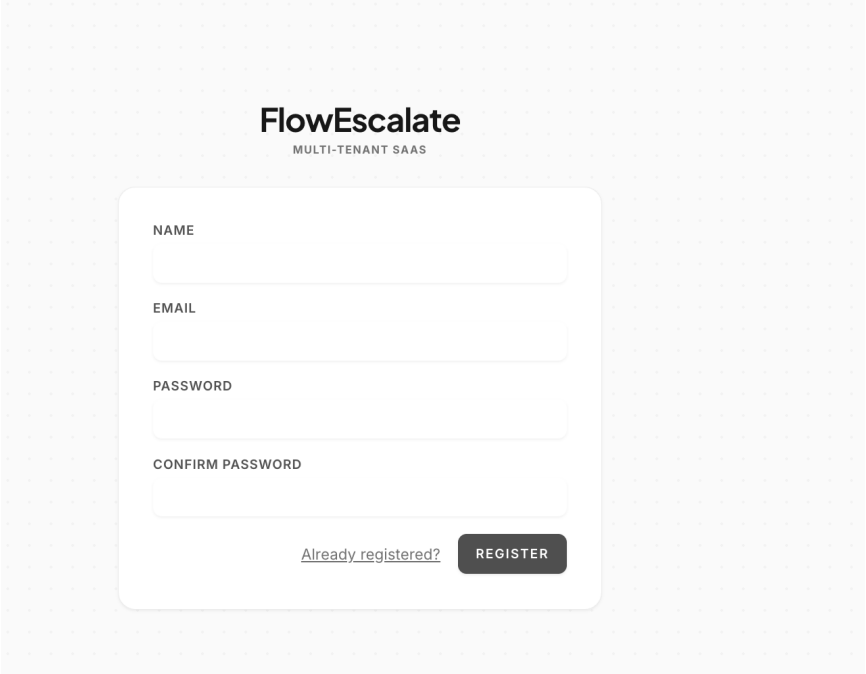

Figure 4.7 — FlowEscalate registration screen

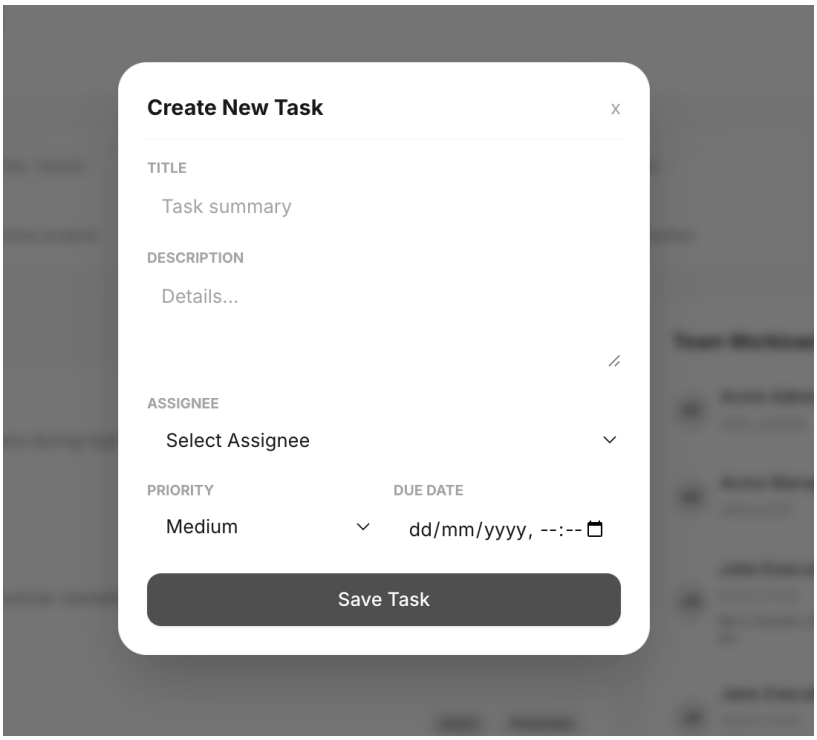

Figure 4.8 — FlowEscalate Create New Task dialog

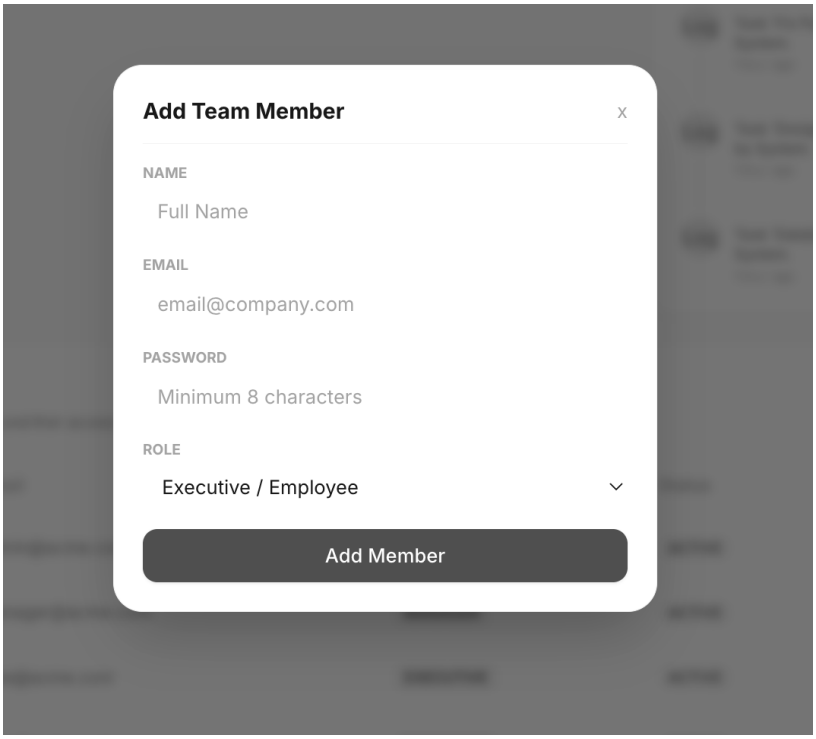

Figure 4.9 — FlowEscalate Add Team Member dialog

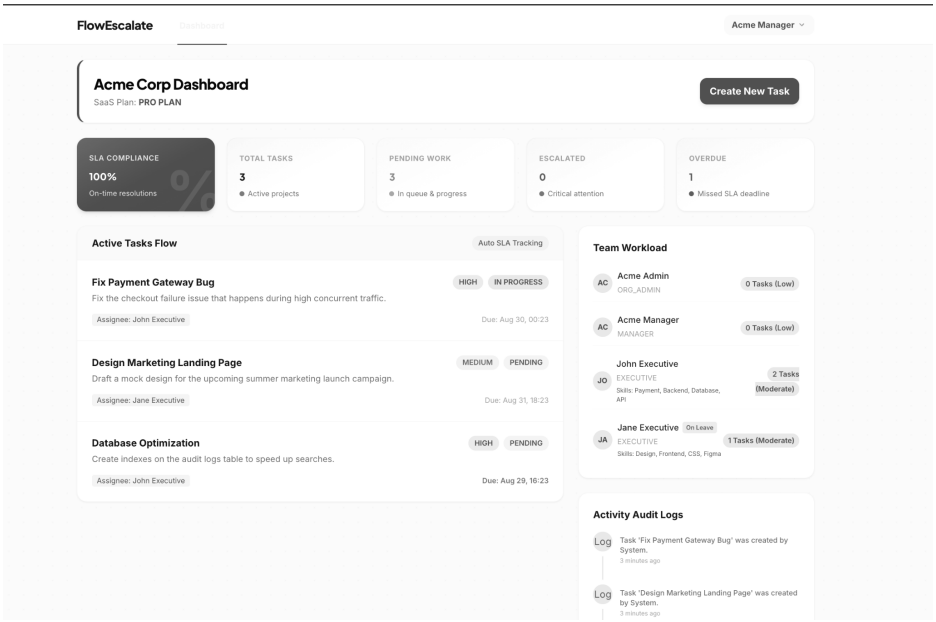

Figure 4.11 — FlowEscalate organization dashboard showing tasks, workload and SLA indicators

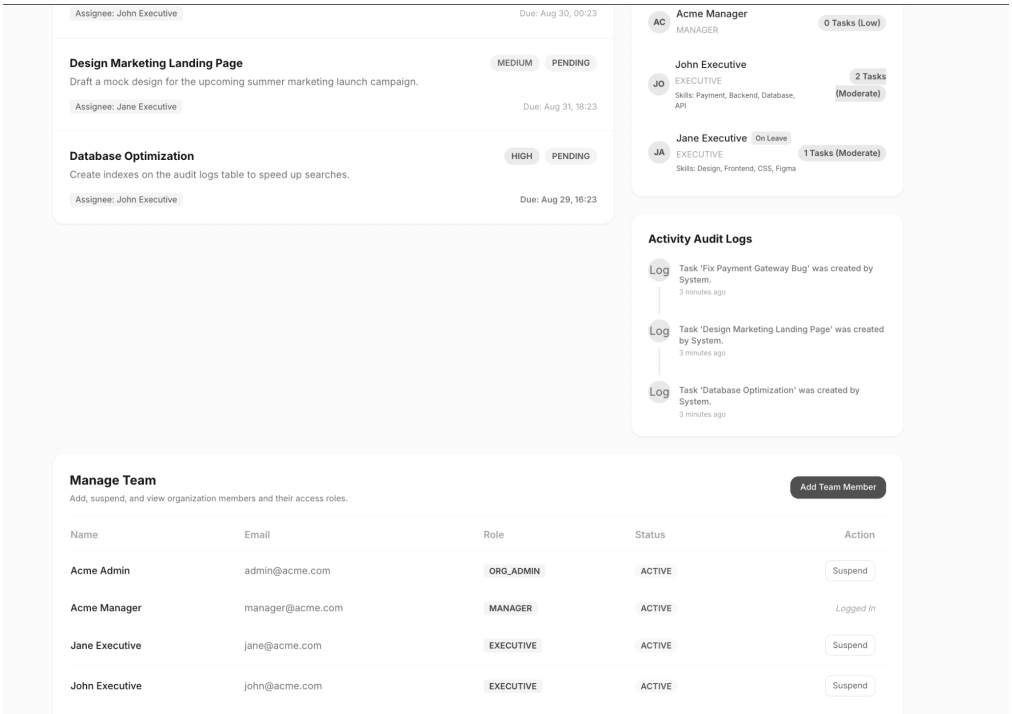

Figure 4.12 — FlowEscalate team management and activity/audit area

4.7.7 FlowEscalate — SLA Monitoring and Escalation

Service Level Agreement monitoring is designed to detect tasks whose deadlines have been breached. The system can identify overdue work, increment an escalation level, and route responsibility through a role hierarchy. This reduces dependence on manual monitoring and provides a structured response to missed deadlines.

The dashboard presents indicators such as SLA compliance, total tasks, pending work, escalated work, and overdue tasks. These indicators help users understand operational status quickly. The audit log complements the dashboard by preserving activity history.

4.7.8 FlowEscalate — Security and Auditability

Security in a multi-tenant application includes both authentication and authorization. Authentication establishes who the user is; authorization determines what that user is allowed to do. Tenant isolation adds another dimension by determining which organization’s data is visible. The project therefore treats tenant context and role permissions as central architectural controls.

Audit logging provides traceability for status changes, assignment changes, SLA-related events, and other important actions. The project also uses closure controls so that completed tasks can be treated as immutable from an operational perspective. These features demonstrate the importance of accountability in enterprise workflow software.

4.8 Methodology

The development and testing methodology followed an iterative workflow: understand the requirement, inspect the existing implementation, implement or modify the relevant component, execute the workflow, identify defects, debug the cause, retest the change, and perform regression checks on related functionality. This approach was used because enterprise applications contain dependencies that make one-time testing insufficient.

For testing tasks, manual execution was a primary technique. Test cases were derived from the expected user journey and from likely failure points. Positive cases verified valid inputs and normal workflows. Negative cases checked invalid inputs, missing information, unauthorized actions, and boundary conditions. Regression

checks were used after fixes to ensure that corrected functionality did not break previously working behaviour.

4.8.1 Software Testing Methodology

Functional testing verifies that each feature performs the intended business function. UI testing checks whether controls, layouts, labels, navigation and feedback behave correctly. Form validation checks required fields, formats, invalid values, and error messages. Authentication testing verifies login and signup behaviour. Product, cart and checkout testing verifies the e-commerce workflow from discovery through transaction preparation.

Responsive testing checks the same workflow across different viewport sizes and device orientations where practical. Performance-oriented testing observes whether pages and interactions remain responsive under ordinary usage. Manual testing is particularly valuable during exploratory work because it allows the tester to follow unexpected paths and notice usability issues that a fixed script may miss.

4.8.2 Representative Test Cases

Table 4.6 — Representative functional test cases

TC ID	Module	Test scenario	Expected result
TC-01	Login	Submit valid credentials	User is authenticated and redirected to the correct dashboard.
TC-02	Login	Submit invalid credentials	Access is denied and an understandable validation/error message is shown.
TC-03	Signup	Submit incomplete registration form	Required validation prevents incorrect submission.
TC-04	Product	Open product from category/home screen	Correct product information is displayed.
TC-05	Cart	Add product to cart	Selected product appears in cart with correct information.
TC-06	Cart	Remove/update cart item	Cart state reflects the requested change.
TC-07	Checkout	Submit missing required information	Validation prevents invalid progression.
TC-08	Responsive	Open key screens on mobile-sized viewport	Content remains accessible and usable.
TC-09	Offline Exam	Create/modify exam schedule	Schedule is saved with valid information and no unintended conflicts.
TC-10	Online Exam	Open configured exam	Authorized user can access the intended exam.
TC-11	Question Bank	Use question-management workflow	Question data is handled according to the configured workflow.
TC-12	Regression	Repeat related workflow after a fix	Existing functionality continues to work.

4.8.3 Bug Reporting and Debugging Process

When an issue was encountered, the first step was reproduction. Reproducing the same failure consistently helps distinguish a real defect from an environmental or user-specific issue. The next step was to record the affected workflow, input conditions, expected result, actual result, and any relevant error information.

Debugging then involved tracing the application path through the relevant UI, route, controller, validation, database interaction, or framework component. After a correction, the original scenario was repeated. Related scenarios were also checked

to reduce the possibility of regression. This process improved the student’s ability to reason about software failures systematically rather than relying on trial-and-error changes.

4.8.4 Examination Scheduling Methodology

Examination scheduling was handled as a controlled data workflow. The process begins with examination configuration, followed by selection of academic context and schedule information. The system must then present consistent information to administrators and, where applicable, students. Changes to a schedule must be validated so that the final state reflects the intended examination arrangement.

The student's responsibility included careful validation of schedule-related information and testing of related workflows. Because examination information is operationally sensitive, the methodology emphasized accuracy, repeatability, and regression checks after modifications.

4.8.5 Online Examination Methodology

Online examination testing was approached as an end-to-end user journey. The examination had to be available to the intended user, questions had to be accessible, the user had to be able to interact with the test, and the final submission had to follow the expected application workflow. Testing also considered the effect of invalid or incomplete conditions.

The focus was on preventing avoidable failures in the online exam experience. The objective was not merely to confirm that an exam page opened, but to ensure that the full process could be completed effectively and without obvious code or workflow errors.

4.8.6 Development Methodology for FlowEscalate

FlowEscalate was developed using an iterative modular approach. The application was divided into tenant management, authentication/authorization, team management, task management, SLA monitoring, audit logging, and dashboard functionality. Backend services and framework features were implemented first or in parallel with corresponding UI components, followed by automated and manual verification.

Unit and feature tests were authored using PHPUnit. Particular attention was given to tenant boundaries because a security defect in multi-tenancy can expose data across organizations. The test strategy therefore included verification that organization-scoped records remain isolated and that role-based permissions prevent unauthorized operations.

4.8.7 Risk and Mitigation Analysis

Table 4.7 — Risk and mitigation analysis

Risk	Potential impact	Mitigation
Incorrect validation	Invalid or incomplete data enters workflow	Validate inputs and test positive/negative cases.
Regression after change	Previously working module breaks	Perform targeted regression checks.
Cross-tenant data exposure	Serious security/privacy failure	Enforce tenant context and organization-scoped queries.
Exam scheduling inconsistency	Incorrect academic operations	Validate schedule inputs and related dependencies.
Online exam failure	Student unable to complete examination	Test complete exam journey and submission workflow.
UI responsiveness issue	Poor usability on mobile/smaller screens	Perform responsive testing across representative sizes.
Incomplete bug reproduction	Incorrect fix or repeated defect	Document exact steps and reproduce before/after fix.

4.8.8 Version Control and Development Discipline

Version control is an important part of professional development because it creates a history of changes and supports safer experimentation. Git and GitHub were part of the student's full-stack learning path. The student learned to think of a feature as a controlled change to a codebase rather than an isolated piece of code.

Good development discipline also includes keeping changes focused, testing before and after modifications, avoiding unnecessary changes to unrelated modules, and documenting important assumptions. These habits reduce debugging time and make it easier to understand why a particular implementation exists.

4.9 Expected Outcomes

The expected outcomes of the internship were both technical and professional. Technically, the student was expected to become comfortable with a modern PHP/Laravel full-stack environment, understand enterprise workflows, perform practical software testing, and contribute to reliable application functionality. Professionally, the student was expected to improve problem-solving, communication, attention to detail, and responsibility for delivered work.

The internship outcomes were achieved through repeated exposure to real application workflows. The student progressed from broad knowledge of programming technologies to deeper practical understanding of how frameworks, databases, interfaces, testing and debugging work together in a software product.

4.9.1 Technical Outcomes

1. Practical understanding of PHP and Laravel based full-stack development.
2. Improved understanding of MVC, routing, controllers, models, validation and views.
3. Experience with Blade, Tailwind CSS, JavaScript and Vite-supported frontend workflows.
4. Hands-on functional and UI testing experience.
5. Experience testing authentication, forms, products, carts and checkout flows.
6. Experience with enterprise examination workflows and scheduling.
7. Improved debugging and regression-testing ability.
8. Exposure to REST-oriented application communication and database-backed systems.
9. Independent implementation experience with multi-tenant SaaS architecture through FlowEscalate.
10. Experience writing PHPUnit unit and feature tests in the independent project.

4.9.2 Professional Outcomes

The internship improved the ability to work with requirements that are not always expressed as a single technical specification. Testing often required understanding what a user or administrator expects to happen and then checking whether the application implements that expectation. This strengthened requirement interpretation and practical decision-making.

The student also learned that accuracy and reliability are part of professional responsibility. This was particularly evident in the school ERP examination modules, where schedule and examination information can affect many users. Careful validation, testing, and retesting became necessary rather than optional.

Communication improved through the need to describe defects clearly, discuss expected behaviour, and report the status of work. The hybrid working environment further encouraged self-management, planning, and maintaining a consistent development/testing routine.

4.9.3 Before Internship vs After Internship

Table 4.8 — Learning outcomes

Area	Before internship	After internship
Programming	Strong academic exposure to Java, Python, C/C++, DSA	Broader practical exposure including PHP/Laravel full-stack development.
Web development	HTML/CSS, JavaScript, React, Spring Boot exposure	Practical Laravel, Blade, Tailwind, Vite and backend workflow experience.
Databases	SQL concepts	Database-backed application workflows and validation.
Testing	Academic/basic testing knowledge	Structured manual functional/UI/regression testing and debugging.
Enterprise systems	Limited practical exposure	Hands-on examination ERP workflows and SaaS architecture.
Security	Conceptual understanding	Practical exposure to authorization and tenant isolation through FlowEscalate.
Software quality	General awareness	Defect reproduction, correction, retesting and regression discipline.

4.9.4 FlowEscalate — Independent Project Results

The independent FlowEscalate project resulted in a working multi-tenant task and SLA workflow demonstration. The supplied screens show separate organizations, distinct user roles, dashboards, task status indicators, team workload, overdue indicators, and audit activity. These screens provide visual evidence of the architecture and workflow concepts implemented in the project.

The project also reinforced the importance of automated testing for security-sensitive logic. PHPUnit unit and feature tests were used to verify application behaviour, including tenant boundaries. This complements the manual testing experience gained during the internship.

4.9.5 Evidence from Internship Applications

The School Admin screen provides evidence that examinations and online exams are first-class modules within the ERP. The quick-action layout makes these workflows directly accessible to administrative users.

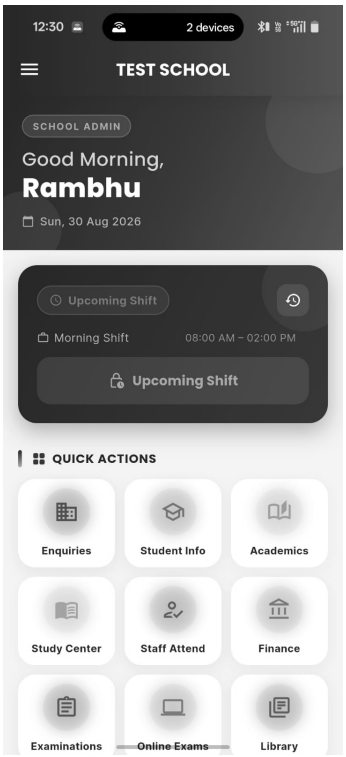

Figure 4.13 — School Admin dashboard showing examination and online-examination entry points

4.9.6 Overall Outcome

The combined internship and independent project experience produced a broader understanding of software engineering. The internship provided exposure to real application testing and enterprise examination workflows, while the independent project provided deeper backend architecture and SaaS security experience. Together, they strengthened the student's ability to work across development, testing, debugging, databases, UI, and application architecture.

The experience also clarified the relationship between development and quality assurance. A developer must understand how a feature can fail; a tester must understand enough of the implementation and business workflow to identify meaningful defects. Working across both perspectives is valuable for full-stack engineering.

4.10 Certificates of Completion and Communication Mail Proof

The internship offer letter is included as documentary evidence of the internship engagement. It records the student's name, university, company, Development department, Full Stack Developer role, joining date, internship term, and signatory details.

The original Internship Completion Certificate will be inserted by the student in the dedicated certificate page reserved earlier in this report. Communication evidence may also be attached in the appendix where appropriate. The supplied onboarding email screenshot shows a communication from Shivansh requesting completion of an onboarding form and identifying the sender as Physics Wallah (FinZ).

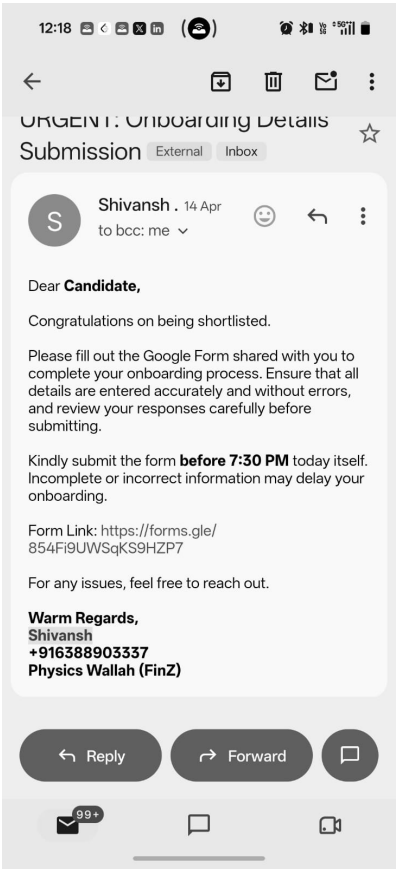

Figure 4.14 — Internship onboarding communication evidence

5. BIBLIOGRAPHY / REFERENCES

1. Physics Wallah (FinZ), Internship Offer Letter issued to RAMBHU SINGH, dated 12-05-2026.

2. IILM University, School of Computer Science and Engineering, “Internship Report Format updated 26-27”.

3. Laravel Documentation, framework concepts including routing, controllers, middleware, validation, database access and testing.

4. PHP Documentation, language reference and web application programming concepts.

5. Tailwind CSS Documentation, utility-first styling and responsive design concepts.

6. Vite Documentation, frontend development and asset bundling concepts.

7. PHPUnit Documentation, unit and feature testing concepts for PHP applications.

8. Git Documentation, version control concepts and repository workflows.

APPENDIX — INTERNSHIP OFFER LETTER

FINZ FINTECH PRIVATE LIMITED

+91-9289310672
support@finz.live
https://finz.live/

INTERNSHIP OFFER LETTER

Date: **12-05-2026**

To
RAMBHU SINGH,
IILM UNIVERSITY GREATER NOIDA

We are pleased to offer you an internship opportunity at FinZ Fintech Private Limited ("Company"/ "PW"), **Corporate - Noida - UP (KLJ Noida One, First Floor, Sector 62, Noida, 201309)** in FinZ. You will be joining the **Development** department in the role of **Full Stack Developer**. Your internship shall commence on **18-05-2026** (**"Joining Date"**) and shall be for a period of **3 months** (**"Internship Term"**)

Compensation
You'll be paid 0**/-** on a monthly basis **("Stipend")**.

You will not be under the direct payroll of the Company and therefore, you will not be receiving any of the employee benefits including, but not limited to, health insurance, etc. that the Company offers to its permanent and full-time employees.

For this internship, your major duties will include, but are not limited to activities and innovation.

It is understood that your internship is voluntary and treated as 'at-will'. Further, please note that your internship will be subject to the correctness of all the information and necessary documents furnished by you. In the event, it is found that any such material information furnished by you, whether verbally or in writing, at any time, is incorrect, misrepresented or fabricated, the Company shall have the right to terminate your internship without any notice or compensation.

During your tenure of internship, you would be governed by the Company policies and any other agreement that you may execute with the Company from time to time.

During your internship, you may have access to confidential, proprietary, and/or trade secret information belonging to the Company. All of this information shall be kept strictly confidential, and you shall not use it for your own purposes or shall not disclose it to anyone outside the Company. Upon conclusion of the internship, you shall immediately return to the Company all of its property, equipment, and documents provided to you during the term of your internship.

Plot No. B-8, Tower A, Noida One, Ground Floor, Sector 62, Noida, UP - 201309

Appendix Figure A.1 — Internship Offer Letter, page 1.

During the term of your internship, you may create work reports, summaries, synopsis, projections among others for the Company, and all such work that is reduced to fixed form or is otherwise capable of any intellectual property protection will be the absolute property of the Company. You agree not to make claims over the same and to further do all acts, deeds or things as the Company may desire to enable the Company to have such rights over the works and or the work products.

As communicated to you at the time of offer, please ensure that effective immediately, you are required to work on your own personal laptop and no laptop shall be provided to you by the company for the purpose of fulfilling your work responsibilities and tasks. Furthermore, failure to comply with this requirement may result in leave without pay being applied for the duration of the non-compliance. Additionally, please note that if you are in non-compliance for a continuous period of 7 days, it may lead to the termination of your internship with the company.

In accordance with the terms of this internship offer letter, the internship period is anticipated to conclude after **3 months** starting from the **18-05-2026**. Nevertheless, should you desire to prematurely terminate your internship, you shall be obligated to provide the company with a written notice of 30 (thirty) days in advance.

To confirm your acceptance, please reply to the offer letter email with the following statement as a token of your acceptance: "I acknowledge all the terms & conditions and company policies, and I accept this offer."

We look forward to welcoming you to the FinZ team.

Yours Sincerely,
FinZ FinTech Pvt. Ltd.

Name: Piyush Jaiswal
Designation: National Head

Appendix Figure A.2 — Internship Offer Letter, page 2.

ACCEPTANCE OF THE INTERNSHIP OFFER LETTER

By signing this Internship Letter and thereby acknowledging and accepting this Internship Letter, you confirm and agree that such acceptance and signature constitutes a valid and legally binding Internship Agreement between you and the Company. You understand that by accepting and signing this Internship Letter, you are entering into a legally enforceable contract, and that you agree to be bound by all the terms and conditions set forth in this Internship Letter and all applicable Company Policies and Procedures.

For and on behalf of the **"Intern"**
Name: RAMBHU SINGH,
Date: 12-05-2026

Appendix Figure A.3 — Internship Offer Letter, page 3.